

Sessions and Guaranteed Outcomes

What a session actually persists, when to resume and when to start fresh, and how to make an agent's work end somewhere every time.

A session is a saved **conversation**. It is not a saved system.

What is in a session

- ◆ **The full message array.** Prompts, responses, tool calls and tool results, in order.
- ◆ **Written to disk automatically,** as a transcript file under the project directory.
- ◆ **Scoped to the working directory.** Lookup is project-aware, so run the command where the session began.
- ◆ **Local to the machine that made it.** A CI worker or fresh container has no idea it exists.

ON THE EXAM

Returning to a session means the agent has the files it read, the analysis it did, and the decisions it made.

Sessions and the filesystem are different things

SESSION

Conversation history

- What the agent said, read and decided
- Resuming restores its memory of the work
- Forking branches the conversation
- Nothing here touches your files

CHECKPOINTING

Filesystem snapshots

- What the agent actually changed on disk
- Rewinding restores the files
- A separate mechanism entirely
- Use it to undo edits, not to undo context

THE TIEBREAKER

If the agent edited a file, that edit is already on disk. Resuming gives it the memory of editing, not the ability to un-edit.

Four ways to start

WHAT YOU WANT	WHAT YOU USE	WHAT YOU GET
Pick up where I left off	<code>--continue</code> , or <code>-c</code>	The most recent session in this directory.
Go back to a particular thread	<code>--resume <id or name></code>	That session, continued. It never creates one.
Try something without losing this	<code>--fork-session</code>	A new session ID branching from that point.
Clean slate	Just run it	Fresh context. Nothing inherited.

ON THE EXAM

Name sessions when you start them. A UUID in a runbook three months later is useless.

What forking is actually for

- ◆ **Two approaches from one position.** Try the risky refactor without abandoning the thread you are on.
- ◆ **The original is untouched.** Different session ID, original history unchanged, both resumable separately.
- ◆ **It branches the conversation, not the filesystem.** The same rule as before applies.
- ◆ **Not a way to save context.** The fork carries the parent's history, so it costs the same to load.

ON THE EXAM

Fork before an experiment. It costs nothing and it preserves the thread you would otherwise have to rebuild.

A resumed session gives you everything from before. Including the parts that are **no longer true.**

What goes stale in a resumed session

- ◆ **File contents read hours ago.** The agent believes the old version because that is what it can see.
- ◆ **Query results and API responses.** Prices, statuses, ticket states, all frozen at fetch time.
- ◆ **Its own conclusions,** drawn from data that has since changed.
- ◆ **Nothing marks any of it as old.** A tool result from Tuesday looks identical to one from a second ago.

ON THE EXAM

The agent cannot tell fresh context from stale context. Only you know how much time passed.

Resume, or start fresh with a summary?

RESUME

Continuity matters more

- Little has changed since you stopped
- The reasoning chain is worth keeping intact
- Short gap, same working state
- You want the detail, not the gist

FRESH PLUS SUMMARY

Accuracy matters more

- The world moved while you were away
- Stale tool results would mislead it
- You inject conclusions, not raw history
- Costs you nuance; buys you correctness

THE TIEBREAKER

Ask what changed while you were gone. Nothing means resume. Something means a fresh session with what you know now.

Writing the summary you inject

- ◆ **Decisions and their reasons.** Not the discussion that produced them.
- ◆ **Current state, stated as current.** What the files look like now, not what they looked like then.
- ◆ **What is still open,** and what was ruled out and why.
- ◆ **Not a transcript.** If you paste the history back in you have reinvented resume, with extra steps.

ON THE EXAM

A good summary is shorter than the history and more accurate than it. That is the whole trade.

Sessions across machines

The detail that bites in CI and in containers.

- ◆ **Transcripts are local files.** A fresh CI worker has none of them.
- ◆ **Containers lose everything on restart** unless the home directory is a mounted volume.
- ◆ **Capture the session ID from JSON output** if you intend to resume programmatically.
- ◆ **Or move the transcript, or attach a session store** so another host can pick it up.
- ◆ **Do not rely on --continue in CI.** Most recent in this directory means nothing on a clean runner.

ON THE EXAM

Ephemeral infrastructure plus session-based memory is a combination that fails silently on the second run.

Now the harder question. Not **how does work resume**, but **how do you know it finished?**

Six ways a run can end

- ◆ **end_turn.** Claude produced no tool calls. The task is complete as far as the model is concerned.
- ◆ **The turn cap fired.** It ran out of iterations mid-task.
- ◆ **The budget cap fired.** It ran out of money mid-task.
- ◆ **An error during execution.** A tool threw, a network call failed.
- ◆ **refusal.** Claude declined. This arrives as a normal successful response, not an error.
- ◆ **The context window filled.** There was nowhere left to put the next turn.

ON THE EXAM

Five of the six leave work unfinished. Code that treats "the loop exited" as "the task succeeded" reports five kinds of failure as success.

The orchestration layer

01

Run the agent

Let it do the work, with caps set.

02

Classify the ending

Which of the six was it?
Read the result, not the prose.

03

Decide

Complete, retry,
resume, or escalate.
Your code chooses.

04

Guarantee an outcome

Every path ends
resolved or handed to
a human. No silent
exits.

ON THE EXAM

This wrapper is what turns "usually finishes" into "always ends somewhere". The agent cannot do it for you.

Where a guarantee can live

IN THE PROMPT

Persuasion

- "Always escalate if you cannot resolve it"
- Works most of the time, which is the trap
- Fails exactly when the agent is confused
- Cannot be audited or tested

IN THE CODE

Enforcement

- A check that runs after every invocation
- Runs whether the agent behaved or not
- Independent of prompting and phrasing
- Testable, and provable to an auditor

THE TIEBREAKER

The word "guarantee" in a stem eliminates every prompt-shaped option before you have read them.

Where enforcement can sit

- ◆ **In the orchestration wrapper.** After the run, classify and act. The most general answer.
- ◆ **Inside the tool.** A refund tool that checks the limit itself cannot be talked past.
- ◆ **In a hook.** Fires on the event whatever the agent intended, and can block.
- ◆ **In the toolset.** A tool the agent does not have is a capability it does not have.
- ◆ **Not in CLAUDE.md, and not in the system prompt.** Both are context. Context persuades.

ON THE EXAM

Four real places, and one that only looks like a place. Domain 2 and Domain 3 return to all of them.

Resuming safely after a failure

The wrapper's other job, and an easy question to get wrong.

- ◆ **A run that failed halfway may have already acted.** The refund went out. The ticket was created.
- ◆ **The session records that it happened.** A blind retry from scratch does it twice.
- ◆ **Resume, so the agent knows what it already did,** rather than restarting the same task cold.
- ◆ **Or make the side-effecting tools idempotent,** so a repeat is harmless.

ON THE EXAM

"Retry the whole task" is a safe answer only when the task has no side effects. Check before you pick it.

Reading the stem: session questions

WHAT THE STEM SAYS	WHAT IT WANTS
"pick up where we left off, nothing has changed"	Resume
"the branch was merged while the agent was idle"	Fresh session, inject a current summary
"try an alternative without losing this thread"	Fork
"runs on a fresh CI container each time"	No session exists. Capture and pass the ID, or a store.
"guarantee every conversation ends resolved or escalated"	Orchestration layer, not a prompt
"the run stopped at the turn cap"	Unfinished work. Classify and act; do not report success.

ON THE EXAM

Session questions are usually decided by one sentence about what happened between the two runs.

Module 4 in five lines

- ◆ A session is conversation history on disk. It is not a snapshot of your filesystem.
- ◆ Continue for the last one, resume for a named one, fork to branch. Resume never creates.
- ◆ Resume when nothing moved. Start fresh with a summary when something did.
- ◆ Five of the six ways a run can end leave work unfinished. Classify the ending; never assume.
- ◆ Guarantees live in code. Prompts, CLAUDE.md and system prompts are context, and context persuades.

ON THE EXAM

That closes Domain 1. Five scenarios, then Domain 2 and tool design.

An agent has been working through a large refactor with an engineer for two hours. The engineer breaks for lunch. While they are away, a colleague merges a branch that changes the signatures of four functions the agent has already read and reasoned about.

The engineer returns and wants to continue. The remaining work depends on those four functions.

What is the most reliable way to continue?

- ☐ A Resume the session, so the agent keeps its full understanding of the refactor
- ☐ B Resume the session and ask the agent to re-read the four files before continuing
- ☐ C Start a fresh session with a summary of decisions made and the current state of the code
- ☐ D Fork the session, so the pre-merge version is preserved for comparison

◆ PAUSE AND DECIDE

What is the most reliable way to continue?

- A

 Resume
Restores four stale file reads and every conclusion drawn from them. Confident and wrong.
- B

 Resume and re-read
The strongest distractor. New reads sit alongside old ones in the same history, and nothing marks which is current.
- C

 Fresh session with a summary
Carries the decisions forward and leaves the stale reads behind. Only one version of the code is present.
- D

 Fork
Forking copies the stale history into a second session. Two copies of the wrong picture.

THE SIGNAL

"While they were away" is the planted phrase. It exists to tell you the context no longer matches reality.

A support agent resolves billing queries. Policy requires that every conversation ends either resolved or escalated to a human, with no exceptions.

The system prompt states this clearly and in strong terms. Production data shows 97% of conversations end correctly. The remaining 3% end with the agent producing a final message and no escalation, leaving the customer with no resolution and no ticket.

Investigation shows these are cases where the agent believed it had resolved the issue but had not.

What most reliably closes the gap?

- | | |
|--|---|
| <p>A Strengthen the system prompt with explicit examples of situations requiring escalation</p> | <p>B Add an orchestration check after each conversation that escalates any interaction not marked resolved</p> |
| <p>C Add a rule to CLAUDE.md requiring escalation whenever the agent is uncertain</p> | <p>D Lower the confidence threshold at which the agent chooses to escalate</p> |

◆ PAUSE AND DECIDE

What most reliably closes the gap?

- A

 Stronger prompt with examples
Might move 97% to 99%. The requirement says no exceptions, and a prompt cannot deliver that.
- B

 Post-run orchestration check
Runs whatever the agent decided. It does not depend on the agent recognising its own failure.
- C

 Rule in CLAUDE.md
A memory file is context, and it is a Claude Code mechanism that is not even present in this deployment.
- D

 Lower the threshold
Relies on the agent knowing it is uncertain. These cases are ones where it wrongly believed it had succeeded.

THE SIGNAL

The failures are cases where the agent thought it had succeeded. Every option asking the agent to judge better is already ruled out.

A nightly job runs an agent that triages the previous day's error reports. Each run happens on a fresh CI container.

The team wants each night's run to remember what it concluded the night before, so recurring errors are recognised rather than re-investigated. They have added `--continue` to the command.

Every run behaves as though it is the first.

Why, and what should they do instead?

- A `--continue` only works interactively. Use `--resume` with the session ID instead
- B The container has no prior session on disk. Persist the transcript or a session store, or carry conclusions forward in a file
- C Sessions expire after 24 hours, so the previous night's session is gone by the time the job runs
- D `--continue` needs the session to have ended cleanly, and CI runs are being terminated by the turn cap

◆ PAUSE AND DECIDE

Why, and what should they do instead?

- A

 Use --resume with the ID
Closer, but incomplete. Resume also reads a local transcript file that this container does not have.
- B

 No session exists on this host
Transcripts are local files. A fresh container starts with nothing, so something must be carried across.
- C

 Sessions expire in 24 hours
Invented. There is no such expiry, and the failure would be intermittent rather than total.
- D

 Unclean termination
Also invented, and it does not explain why the very first resumed run behaves as a first run.

THE SIGNAL

"A fresh CI container" is the whole question. Session state is a file on a machine, and this machine is new every night.

An agent processes supplier invoices. It validates the invoice, records it in the ledger, and issues payment. Payment is not idempotent.

One run failed partway: the invoice was validated and recorded, then a network error interrupted the run before payment. The orchestration layer's retry logic starts the task again from the beginning with the same input.

The team is now investigating duplicate ledger entries.

What is the most appropriate change?

- | | |
|---|--|
| <p>A Add a longer timeout so network errors are less likely to interrupt a run</p> | <p>B Resume the failed session rather than restarting, and make the side-effecting tools idempotent</p> |
| <p>C Wrap the whole task in a database transaction that rolls back on failure</p> | <p>D Have the agent check for an existing ledger entry before recording one</p> |

◆ PAUSE AND DECIDE

What is the most appropriate change?

- A

 Longer timeout
*Fewer interruptions, same behaviour when one happens.
Reduces frequency, not consequence.*
- B

 Resume plus idempotent tools
Resuming tells the agent what it already did. Idempotency covers the cases where resuming is not possible.
- C

 Wrap in a transaction
The payment call is an external side effect. No database transaction can roll that back.
- D

 Agent checks first
Makes correctness depend on the agent remembering to check. It is the right idea in the wrong layer.

THE SIGNAL

Two answers are right-shaped. D puts the check in the agent; B puts it in the tool. Enforcement belongs below the model.

An agent performs overnight data reconciliation. It is invoked with a turn cap of 30 and a budget cap.

The orchestration wrapper logs a run as successful whenever the invocation returns without throwing an exception, then marks the reconciliation complete in the tracking system.

An auditor has found reconciliations marked complete where roughly a third of the records were never examined.

What is the root cause?

- ☐ A The turn cap is too low for the volume of records
- ☐ B The wrapper treats any non-exception return as success, including runs aborted by the caps
- ☐ C The agent should be instructed to report when it has not finished
- ☐ D Reconciliation should be split across several smaller invocations

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Turn cap too low
Contributing, and the real fault is that hitting it was recorded as success. Raise it and the same silent failure returns at a larger volume.
- B

 Non-exception treated as success
A cap firing is a normal return, not an exception. The wrapper cannot distinguish finished from stopped.
- C

 Instruct the agent to report
A run killed by a cap has no turn left in which to report anything.
- D

 Split into smaller runs
Reduces how often a cap is hit. Each smaller run still reports a capped abort as a success.

THE SIGNAL

Hitting a guardrail is not an error and not a completion. It is a third thing, and the wrapper has to know about it.